

Лабораторна робота № 2

Варіант 7

```
In [9]: class GaussMethodSolver:
    def __init__(self, matrix_a, vector_b):
        """Ініціалізація розв'язувача, створення розширеної
        матриці"""
        self.n = len(matrix_a)
        # Створюємо розширену матрицю (додаємо стовпець вільних
        членів b)
        self.matrix = [row[:] + [b] for row, b in zip(matrix_a,
        vector_b)]
        # Зберігаємо копію початкової матриці для перевірки
        нев'язки
        self.original_matrix = [row[:] for row in self.matrix]
        self.roots = [0.0] * self.n

    def print_state(self, title):
        """Виведення матриці з 6 значущими цифрами"""
        print(f"\n=== {title} ===")
        for row in self.matrix:
            formatted_row = " | ".join(f"{val:10.6f}" for val in
            row)
            print(f"[{formatted_row}]")

    def forward_pass(self):
        """Прямий хід: приведення матриці до верхнього трикутного
        вигляду"""
        self.print_state("Початкова розширена матриця")

        for k in range(self.n - 1):
            for i in range(k + 1, self.n):
                if self.matrix[k][k] == 0:
                    raise ValueError("Головний елемент дорівнює
                    нулю")

                # Обчислюємо множник для поточного рядка
                multiplier = self.matrix[i][k] / self.matrix[k][k]

                # Віднімаємо відповідні елементи
                for j in range(k, self.n + 1):
                    self.matrix[i][j] -= multiplier *
                    self.matrix[k][j]

            self.print_state(f"Матриця після кроку прямого ходу №
            {k + 1}")

    def backward_pass(self):
        """Зворотний хід: знаходження невідомих"""
```

```
for i in range(self.n - 1, -1, -1):
    known_sum = sum(self.matrix[i][j] * self.roots[j] for
j in range(i + 1, self.n))
    self.roots[i] = (self.matrix[i][self.n] - known_sum) /
self.matrix[i][i]

print("\n=== Знайдені корені рівняння ===")
for i, root in enumerate(self.roots):
    print(f"x_{i+1} = {root:.6f}")

def calculate_residuals(self):
    """Обчислення вектора нев'язки  $r = |b - Ax|$ ."""
    print("\n=== Вектор нев'язки (похибка) ===")
    for i in range(self.n):
        # Обчислюємо суму  $A \cdot x$  для i-того рядка початкової
матриці
        ax_sum = sum(self.original_matrix[i][j] *
self.roots[j] for j in range(self.n))
        residual = abs(self.original_matrix[i][self.n] -
ax_sum)
        # Виводимо в експоненціальному форматі, оскільки числа
дуже близькі до нуля
        print(f"r_{i+1} = {residual:.6e}")
```

```

In
[10]: # Матриця коефіцієнтів A
A = [
    [7.03, 1.22, 0.85, 1.135, -0.81],
    [0.98, 3.39, 1.30, -1.63, 0.57],
    [1.09, -2.46, 6.21, 2.10, 1.033],
    [1.345, 0.16, 2.10, 5.33, -12.00],
    [1.29, -1.23, -0.767, 6.00, 1.00]
]

# Вектор вільних членів b
B = [2.1, 0.84, 2.58, 11.96, -1.47]

# Створення об'єкта та запуск методів
solver = GaussMethodSolver(A, B)
solver.forward_pass()
solver.backward_pass()
solver.calculate_residuals()

```

=== Початкова розширена матриця ===

```

[ 7.030000 | 1.220000 | 0.850000 | 1.135000 |
-0.810000 | 2.100000]
[ 0.980000 | 3.390000 | 1.300000 | -1.630000 |
0.570000 | 0.840000]
[ 1.090000 | -2.460000 | 6.210000 | 2.100000 |
1.033000 | 2.580000]
[ 1.345000 | 0.160000 | 2.100000 | 5.330000 |
-12.000000 | 11.960000]
[ 1.290000 | -1.230000 | -0.767000 | 6.000000 |
1.000000 | -1.470000]

```

=== Матриця після кроку прямого ходу №1 ===

```

[ 7.030000 | 1.220000 | 0.850000 | 1.135000 |
-0.810000 | 2.100000]
[ 0.000000 | 3.219929 | 1.181508 | -1.788222 |
0.682916 | 0.547255]
[ 0.000000 | -2.649161 | 6.078208 | 1.924018 |
1.158590 | 2.254395]
[ 0.000000 | -0.073414 | 1.937376 | 5.112849 |
-11.845028 | 11.558222]
[ 0.000000 | -1.453869 | -0.922974 | 5.791728 |
1.148634 | -1.855349]

```

=== Матриця після кроку прямого ходу №2 ===

```

[ 7.030000 | 1.220000 | 0.850000 | 1.135000 |
-0.810000 | 2.100000]
[ 0.000000 | 3.219929 | 1.181508 | -1.788222 |
0.682916 | 0.547255]
[ 0.000000 | 0.000000 | 7.050280 | 0.452779 |
1.720452 | 2.704643]
[ 0.000000 | 0.000000 | 1.964314 | 5.072077 |
-11.829458 | 11.570699]
[ 0.000000 | 0.000000 | -0.389497 | 4.984307 |
1.456986 | -1.608251]

```

=== Матриця після кроку прямого ходу №3 ===

```
[ 7.030000 | 1.220000 | 0.850000 | 1.135000 |  
-0.810000 | 2.100000]  
[ 0.000000 | 3.219929 | 1.181508 | -1.788222 |  
0.682916 | 0.547255]  
[ 0.000000 | 0.000000 | 7.050280 | 0.452779 |  
1.720452 | 2.704643]  
[ 0.000000 | 0.000000 | 0.000000 | 4.945926 |  
-12.308802 | 10.817145]  
[ 0.000000 | 0.000000 | 0.000000 | 5.009321 |  
1.552034 | -1.458831]
```

=== Матриця після кроку прямого ходу №4 ===

```
[ 7.030000 | 1.220000 | 0.850000 | 1.135000 |  
-0.810000 | 2.100000]  
[ 0.000000 | 3.219929 | 1.181508 | -1.788222 |  
0.682916 | 0.547255]  
[ 0.000000 | 0.000000 | 7.050280 | 0.452779 |  
1.720452 | 2.704643]  
[ 0.000000 | 0.000000 | 0.000000 | 4.945926 |  
-12.308802 | 10.817145]  
[ 0.000000 | 0.000000 | 0.000000 | 0.000000 |  
14.018603 | -12.414625]
```

=== Знайдені корені рівняння ===

```
x_1 = 0.104550  
x_2 = 0.127969  
x_3 = 0.600809  
x_4 = -0.016844  
x_5 = -0.885582
```

=== Вектор нев'язки (похибка) ===

```
r_1 = 0.000000e+00  
r_2 = 2.220446e-16  
r_3 = 8.881784e-16  
r_4 = 1.776357e-15  
r_5 = 1.110223e-15
```